

Jaypee Institute of Information Technology
Sector 62



SOFTWARE ENGINEERING LAB [15B17CI573]

SRS DOCUMENT

Title: “***CAMPUS RECRUITMENT SYSTEM***”

Submitted To:-

Dr. Shweta Rani & Dr. Varun Srivastava

Assistant Professor (Senior Grade)

Dept. Of CSE

JIIT SECTOR 62,NOIDA

TEAM MEMBERS

Name	Enrollment No.	Batch
SABEEH AHSAN	22803006	B12
TANMAY BUTTA	22803014	B12
ARYAN KHANNA	22803028	B12

Table of Contents

1. Introduction	
1.1 Purpose	4
1.2 Document Conventions	4
1.3 Intended Audience and Reading Suggestions	4
1.4 Product Scope	4
2. Overall Description	
2.1 Product Perspective	5
2.2 Product Functions	6
2.3 User Classes and Characteristics	7
2.4 Operating Environment	10
2.5 Design and Implementation Constraints	11
2.6 User Documentation	12
2.7 Assumptions and Dependencies	12
3. External Interface Requirements	
3.1 User Interfaces	12
3.2 Hardware Interfaces	16
3.3 Software Interfaces	16
3.4 Communications Interfaces	16
4. System Features	
4.1 System Feature	16
4.2 Functional Requirements	17
5. Other Nonfunctional Requirements	
5.1 Performance Requirements	17
5.2 Safety Requirements	18
5.3 Security Requirements	18
5.4 Software Quality Attributes	18
5.5 Business Rules	18
6. Code Implementation	
6.1 Code Module Implementation & Refactoring	19
7 Software Testing	
7.1 Testing Approaches & Methodologies	24
7.2 Performing Testing on various test cases	25
8 Conclusion	27
9 References	28

1. Introduction

1.1 Purpose

The primary purpose to develop this system is to optimize the recruitment process for college. Besides, the qualified applicants could be sort by this system based on their qualifications and company requirements. Based on the applicants skills and areas of interest , the company suitable or the company in which he/she is going to place can be predicted. Another purpose of the software is to facilitate the student (in the college) and the company to register and communicate with placement office.

1.2 Document Conventions

The font used in this document is Liberation Sans having size 12. The text is made bold for the highlighting purpose so that it can be easily differentiated. In this document, Every functionality is equally important and every requirement has its own priority.

1.3 Intended Audience and Reading Suggestions

The document is meant for the student of the college, the company and the admin will operate the system. This document will serve as a reference document for the Project management and Development team who will analyze, Design and Implement the system. They will coordinate every activity that take place in the software engineering process and will be guided by Dr. Shweta Rani and Dr. Varun Srivastava and other staff of IIIT Computer science & Engineering Department under development.

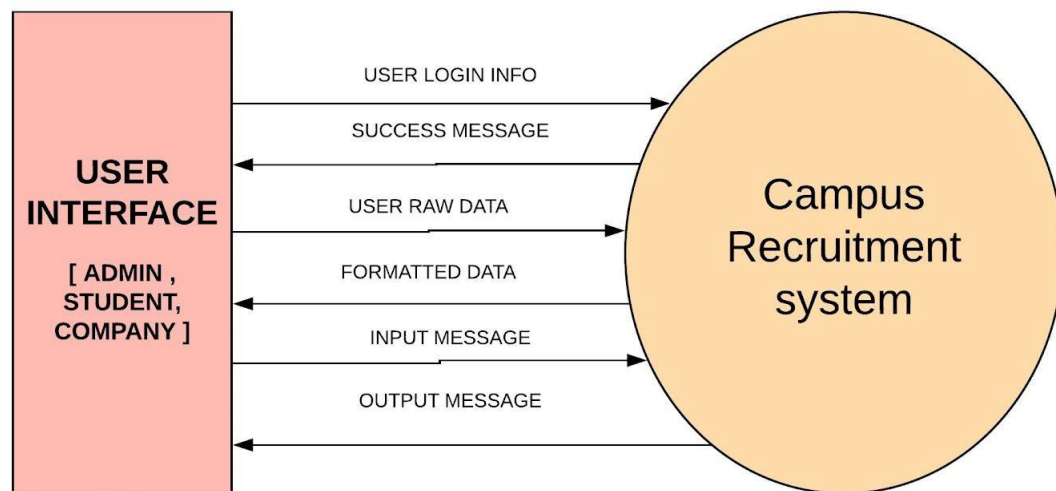
1.4 Product Scope

Campus Recruitment System enables the user to have the typical recruitment facilities and features at their disposal. It resolves the typical issue of manual staffing processes and activities into a controlled and closely monitored work flow in the architecture of the application. The objective of this application is to serve as a common meeting ground for jobseekers and company, locally. This kind of system is specifically designed for organization to help in solving staffing problems and managing human resource department activities at higher degree of optimization.

2. Overall Description

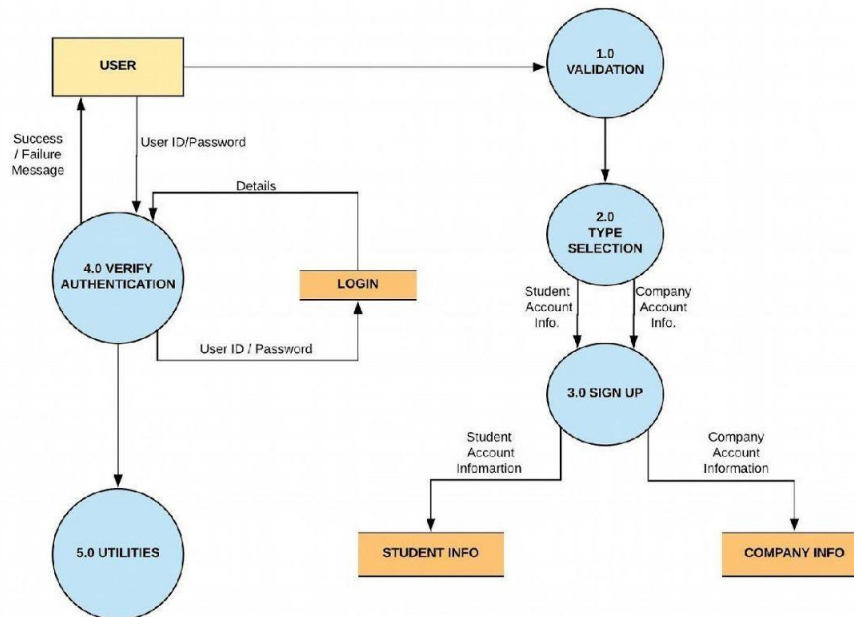
2.1 Product Perspective

In the “Campus Recruitment System”, We will be having a user interface for the admin, student and company. The user can log in to their account. Depending on the user, the utilities will be decided. The student can login to their respective account and can apply for the respective



post available. If the user don't have an account then he/she can create an account and can apply or the jobs. The company can login to their account and can update the details of the jobs available. If the company don't have an account the they can create and account and can update the job details. Admin will already be having an ID and password. The admin will login and manage the student and companies activities. After the creation or the login of the user, a successful message will be displayed to the user.

2.2 Product Functions



**LEVEL - 1 DFD
OF CAMPUS
RECRUITMENT
SYSTEM**

The above DFD clearly shows various various process associated with the project and how the data flows between the entities and database.

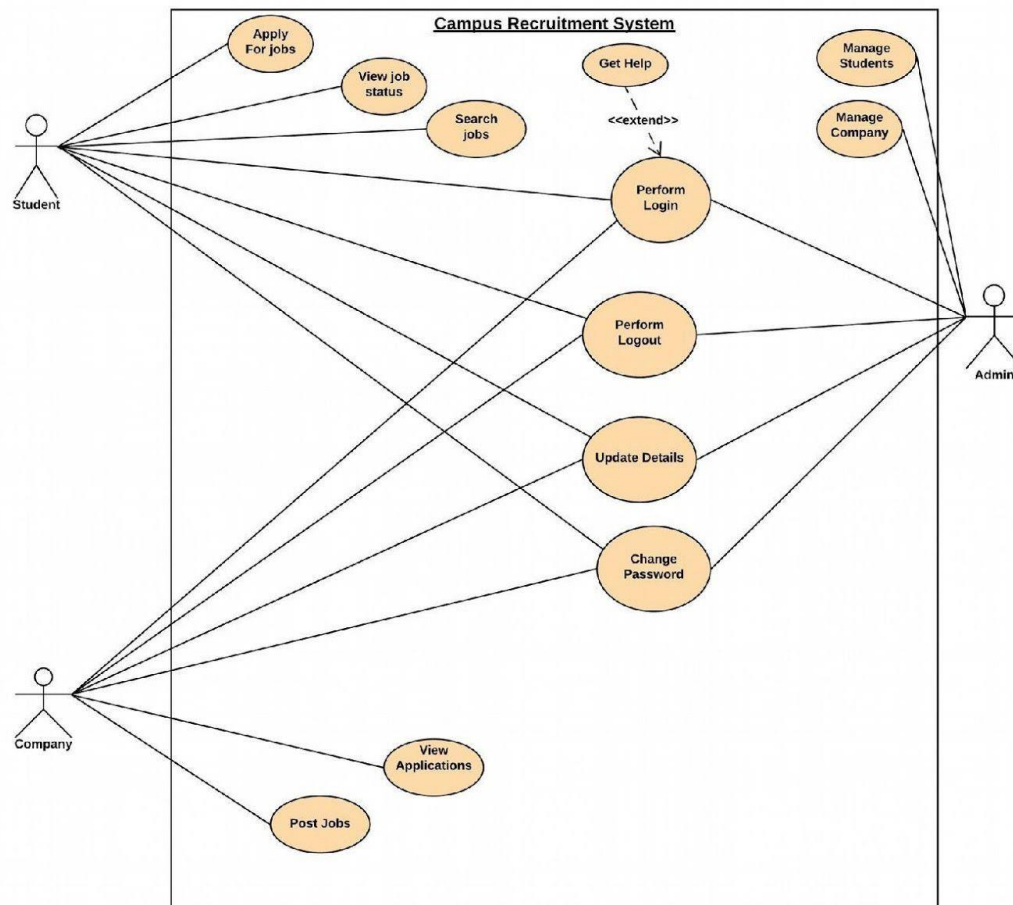
The student can create the account . If the account is already present then he/she can search for the available jobs, can view the jobs and also apply for the jobs. Further he/she can update the details, can change the password etc.

The company can create the account. If the company already have an account then the company can post the job and can also view how many candidates have applied for the jobs of that company.

The admin can manage the student and the company . Admin validates the account of the student . Admin validates the account of the company. Admin can also remove company and the student if found any discrepancy. Admin manages all activities of company and student.

2.3 User Classes and Characteristics

The actors and use cases are clearly shown in below use-case diagram.



In software and systems engineering, a **use case** is a list of actions or event steps typically defining the interactions between a role (known in the Unified Modeling Language as an *actor*) and a system to achieve a goal. The actor can be a human or other external system. In systems engineering use cases are used at a higher level than within software engineering often representing missions or stakeholder goals.

The following are the actors who perform the use cases as stated above:

S.No	Actor Name	Description / Actor's Role
01	ADMIN	Manage students & companies.
02	STUDENT	Apply for jobs, view job status & search jobs.
03	COMPANY	Post jobs & view applications of students.

The following section describes the Use Cases with Pre and Post Conditions:

S.No	Use case Name	Description	Pre-condition	Post condition
01	Perform Login	User(admin,student or company) can perform login.	User having account	Login successful
02	Perform Logout	User(admin, student or company) can perform logout.	User logged in	Successfully logged out
03	Update Details	User(admin, student or company) can update their details.	User logged in	View details
04	Change Password	User(admin, student or company) can change passwords.	User having password.	Password is reset.
05	Manage Company	Admin can manage companies	Existing admin & atleast one company	Validated by admin & company can continue with it's account
06	Manage Students	Admin can manage students	Existing admin & at least one student	Validated by admin & student can continue with it's account
07	Apply for jobs	Student can apply for job	Job should be there posted by company & student must be eligible for job	Applied for job successfully & wait for response from company
08	View Job Status	Student can view job status	Student must have successfully applied for job.	Viewed job status & can accept job if selected else can apply for other job.
09	Search jobs	Student can search for job	Student must have account & logged in.	Can find a job or not.
10	Post Jobs	Company can post jobs	Company must have account & logged in.	Student can now apply for jobs.
11	View Applications	Company can view applications of students.	Atleast one student must have applied.	Company can react to applications.
12	Get Help	User can get help for login	User should have tried for it or just get help if does not know how to login	User will now login using this help.

2.4 Operating Environment

The campus recruitment system shall be deployed on an Operating System Version (windows or Mac). Python as a programming language is used, Visual studio code has been used as code editor and Streamlit is used for frontend.

The system will be built with the help of following softwares:

Sr. No.	System	Development Environment	Description
1.	Campus recruitment System	IDE (Visual Studio)	• Visual Studio is the official Integrated Development Environment (IDE) for web app development
2.	Campus Recruitment system	Python (Programming Language)	Python was created by Guido van Rossum, and first released on February 20, 1991
3.	Campus Recruitment system	Streamlit(for UI)	Streamlit is an open-source Python framework for data scientists and AI/ML engineers to deliver interactive data apps – in only a few lines of code.

2.5 Design and Implementation Constraints

The following Design and Implementation Constraints are applicable for the Campus Recruitment System:

1. The system is designed to be the cross platform supportable. The system is supported on a wide range of hardware and any android platform which is having any version of android built into the system. This application is being developed using visual studio; hence it is extremely portable
2. To prevent multiple students of the same speciality to log-in onto same company portal.
Request, a workflow system needs to be designed which routes the company vacancy Requests to students.
3. System is expected to store maximum 64GB of data.
4. Initially system will be available on android system with versions greater than 5.1. Then the system will be available for even ios mobiles and even on PCs and Laptops.
5. In order to assist students for selecting a company and preparing for interview for that company , a machine learning algorithm will be designed and trained on a training data-set to predict which skills will be required for students and also if student is eligible for company, this algorithm will continue to be trained on previous recorded data sets of students to improve the quality of predictions.
6. The database shall be maintained by admin and who have not logged in for last 1 year would get archived onto a parallel database. Restoring of students data is beyond the scope of the project and would need to be managed by admin.
7. As the system is supposed to be used by students and company as well, care needs to be taken from a usability perspective in terms of font sizes and ease of system usage.
8. Also UI is made with particular animations so that company can find it good for uploading jobs and interact with students. Even more and more companies and students use this app is the aim.

2.6 User Documentation

The following documents shall be prepared:

1. Installation Guide
2. User Manual for end users
3. Even video tutorials of installation of campus recruitment will be provided as DVD.

2.7 Assumptions and Dependencies

The Key Assumptions are:

1. The services is offered for only JIIT students. So right now whole system is designed based on only one college.
2. There will be one authorised student for a particular id. So id will be unique.
3. There will be one authorised company for a particular id. So all ids will be unique.
4. Student can fill maximum 30 application form for jobs in a company.

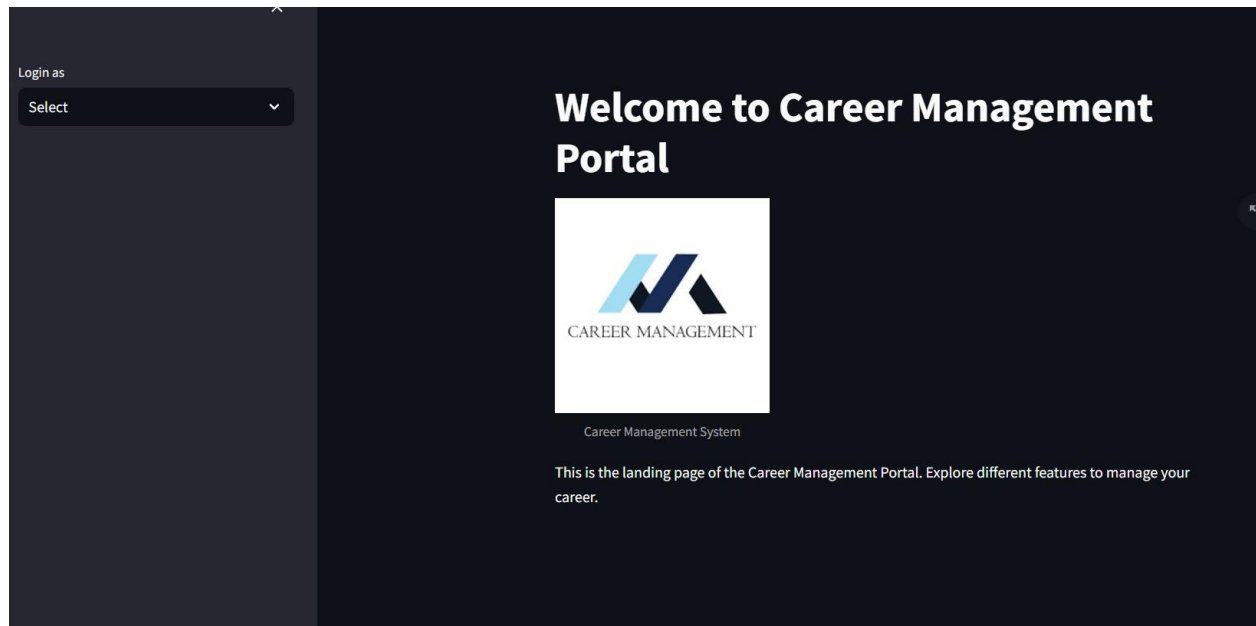
The services can be qualified in terms of volumes of data, trends, frequency of updating in order to give an introduction to the technical system.

3. External Interface Requirements

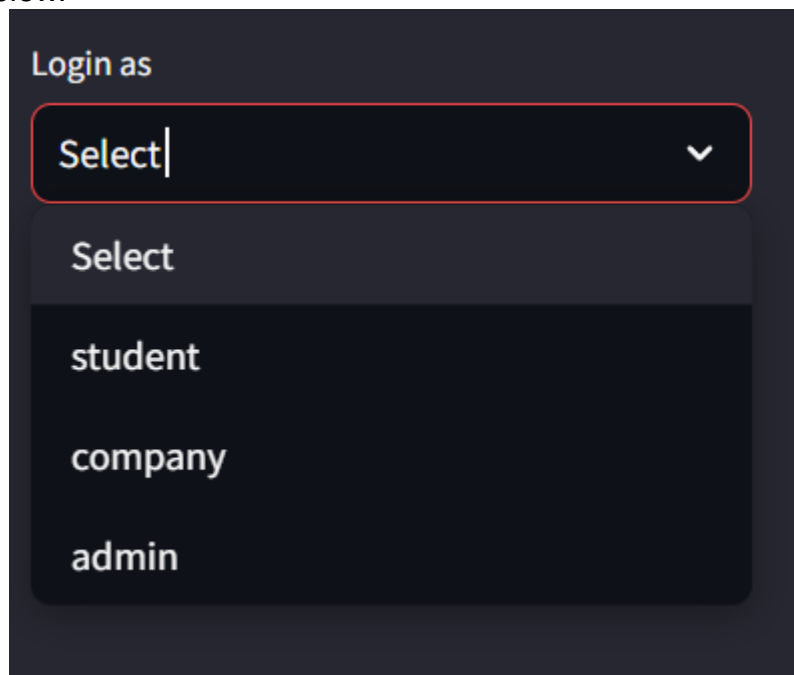
3.1 User Interfaces

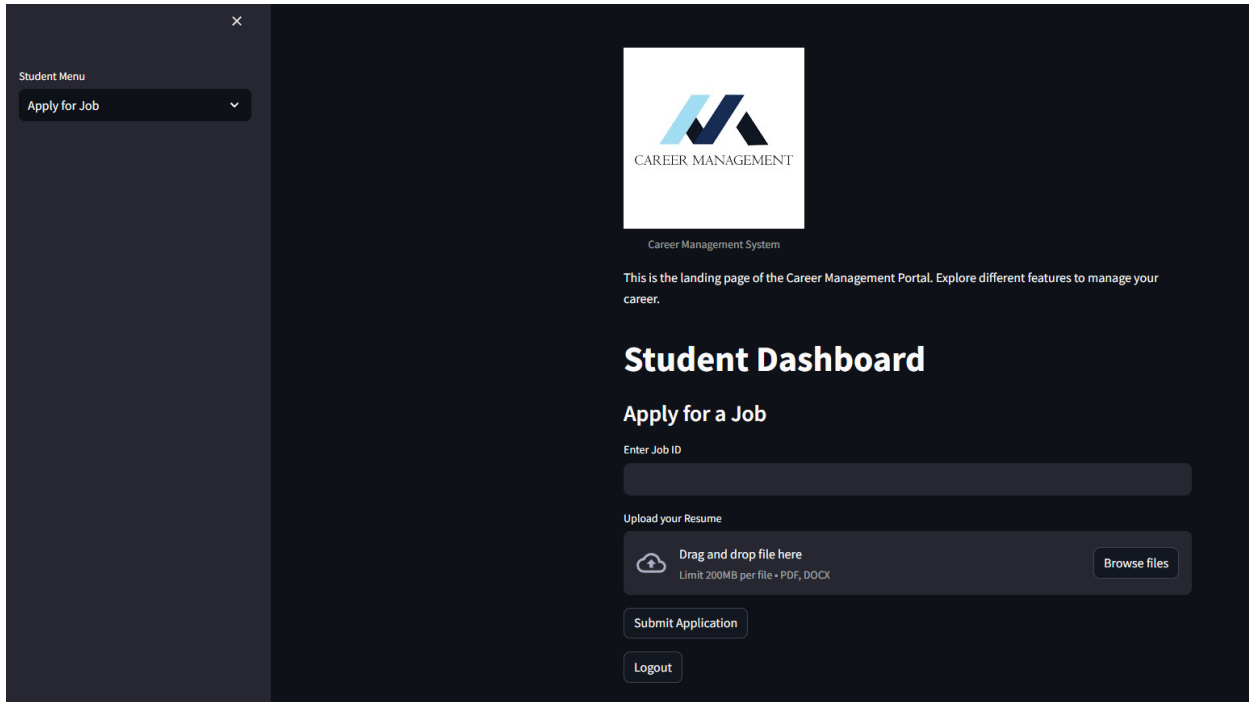
The user interface section defines the way various stakeholders interact with the system. All the screens will be developed to work on web application

A first time user of the mobile should see the login screen when he/she will open the android application. If the user has not registered to the, then he/she should be able to redirect to the sign up page from login screen. Every user should have the profile where he/she can apply for the job. After the creation of account the user can login to the application and will be able to apply for the jobs.



After performing login the user/student will be able to see the side menu as shown in the figure below.





Student Dashboard

Your Job Applications

	Job ID	Company	Status
0	101	ABC Corp	Pending
1	102	XYZ Ltd	Accepted

After performing login by the company the company will be able to see side menu shown in the second figure below.



Company Dashboard



Applications Received

	Student Name	Job ID	Resume Link	Status
0	Tanmay	101	link1.pdf	Pending
1	Aisha	101	link2.pdf	Reviewed

After performing login by the admin the admin will be able to see side menu shown in the figure below.



Admin Dashboard



Manage Students

	Student Name	Roll Number	Email	Status
0	Tanmay	S101	tanmay@example.com	Active
1	Aisha	S102	aisha@example.com	Inactive

Select Action

Add Student



Student Name

Email

Status

Active



Add Student

Logout

User can select the particular section and can perform the respective task. There is an option to add the jobs for the company where the companies can add the vacancies for their companies they have.

3.2 Hardware Interfaces

Since the mobile application does not have designated hardware, it does not have any hardware interface. The hardware connection between the database server and application is managed by the underlying operating system on the mobile phone.

3.3 Software Interfaces

The system is self-contained and no data is supposed to share with the third party. The communication of the mobile application between the database consist of both reading and modifying the data, while the communication between the database and the mobile application consists of only reading operation.

3.4 Communications Interfaces

The communication between the different parts of the system is important since they depends on each other. However, in what way the communication is achieved is not important for the system and is therefore handled by the underlying operating system for the mobile application.

4. System Features

4.1 System Feature and Priority Matrix

Following given is the system features and their priority matrix:

Sr. No.	Feature	Priority
1.	Create an Account	High
2.	Log in	High
3.	Apply for Jobs	Medium
4.	View job status	Low
5.	Search jobs	Low
6.	Change Password	Low
7.	Manage student	High
8.	Manage Company	High
9.	Post jobs	High
10.	View Application	Medium

4.2 Functional Requirements

Following table describes the functional requirement:

Feature	Remark
Update Information	This functionality will update the information of the student.
Change Password	This functionality is used to change the password of account.
Sign Up	This Functionality is used to sign up for student, admin and company.
Log In	This functionality is used to Login for admin, student and company.
Apply For Jobs	This functionality is used for students to apply for jobs.
Post Jobs / Add Job Details	This functionality is used by the company to post jobs.
View Job Status	This functionality is used to view job status by the student.
Search Jobs	This is the functionality which is used to search for jobs.
Manage Student	This functionality is used by the admin to manage student.
Manage Company	This functionality is used by the admin to manage company.
View Student Application	This functionality is used to view student student application.

5. Other Nonfunctional Requirements

5.1 Performance Requirements

1. The completely separate business logic at admin side from the student interface ensures good performance.
2. The system exhibits high performance because it is well optimized. The business logic is clearly separate from the UI.
3. System should be able to scale to many users concurrently.

4. The response time of processes is as follows:

Student Registration max 10 seconds
Company Registration max 15 seconds
Company Job posting max 15 seconds
Student applying for job max 20 seconds

5. System is available 24 by 7.

5.2 Safety Requirements

- Errors will be minimized and an appropriate error message that guides the user from an error will be provided.
- Validation of users input is highly essential.
- The time taken to recover from the error is less than 10 second.

5.3 Security Requirements

1. The system is provided a high level of security and integrity of the data held by the system.
2. only authorized personnel such as admin can gain access to the to the private data and only the user with valid username and password is allowed to view its user page.

5.4 Software Quality Attributes

1. The key software quality attributes are Availability, Reliability and usability.
2. As the system is expected to be 24/7 working. High availability is important.
3. A simple but quality user interface is developed to make it easy to understand and required less training.
4. The error message displayed is more descriptive and can be easily understood.

5.5 Business Rules

- System shall be available only for the particular college.
- All the users shall access the system using a login/user-id and password. The login-id/password will be managed in a secured manner.
- Each student can get recruitment only in one company.
- Once company selected the candidate, it cannot be rejected otherwise company will be blacklisted.
- Each student can have only one account.

6. Code Implementation and Refactoring

This section outlines the core software modules implemented as part of the Campus Recruitment System, followed by their refactored versions. The focus of refactoring is to improve code structure, readability, and maintainability using standard object-oriented practices.

6.1 Core Module Implementations & Refactoring

The following core functionalities have been implemented and refactored in python:

a. User Identification Module

Handles secure login for admins, students, and companies.

```
import streamlit as st

# Dummy credentials for roles
users = {
    "student": {"username": "student", "password": "student123"},
    "company": {"username": "company", "password": "company123"},
    "admin": {"username": "admin", "password": "admin123"}
}

# Landing page
def landing_page():
    st.title("Welcome to Career Management Portal")
    st.image("career.jpg", caption="Career Management System")
    st.write("This is the landing page of the Career Management Portal. Explore different features to manage your career.")

# Initialize session state
if "logged_in" not in st.session_state:
    st.session_state.logged_in = False
    st.session_state.role = None
```

```

def login_page(role):

    st.title(f"{role.capitalize()} Login")

    username = st.text_input("Username")

    password = st.text_input("Password", type="password")

    if st.button("Login"):

        if username == users[role]["username"] and password == users[role]["password"]:

            st.session_state.logged_in = True

            st.session_state.role = role

            st.success("Login successful!")

            st.rerun()

        else:

            st.error("Invalid credentials!")

# Role-based redirection

if not st.session_state.logged_in:

    landing_page()

    role = st.sidebar.selectbox("Login as", ["Select", "student", "company", "admin"])

    if role != "Select":

        login_page(role)

```

b. Application Tracking Module

Allows companies to post job listings:

--- Application Tracking for Students ---

```
def student_page():
    st.title("Student Dashboard")
    menu = st.sidebar.selectbox("Student Menu", [
        "Apply for Job", "View Job Status", "Search Job", "Update Details", "Change Password"
    ])
    if menu == "Apply for Job":
        st.subheader("Apply for a Job")
        job_id = st.text_input("Enter Job ID")
        resume = st.file_uploader("Upload your Resume", type=["pdf", "docx"])
        if st.button("Submit Application"):
            if job_id and resume:
                st.success(f"Applied to job ID {job_id} successfully!")
            else:
                st.warning("Please enter job ID and upload resume.")
    elif menu == "View Job Status":
        st.subheader("Your Job Applications")
        st.table({
            "Job ID": ["101", "102"],
            "Company": ["ABC Corp", "XYZ Ltd"],
            "Status": ["Pending", "Accepted"]
        })
```

```
elif menu == "Search Job":  
    keyword = st.text_input("Search by keyword (e.g., developer)")  
    location = st.text_input("Location")  
    if st.button("Search"):  
        st.info(f"Jobs for '{keyword}' in '{location}'")  
        st.write("1. Frontend Developer - ABC Corp")  
        st.write("2. Data Analyst - XYZ Ltd")  
  
elif menu == "Update Details":  
    name = st.text_input("Full Name")  
    email = st.text_input("Email")  
    phone = st.text_input("Phone Number")  
    if st.button("Update"):  
        st.success("Details updated successfully!")  
  
elif menu == "Change Password":  
    old_pwd = st.text_input("Old Password", type="password")  
    new_pwd = st.text_input("New Password", type="password")  
    confirm_pwd = st.text_input("Confirm New Password", type="password")  
    if st.button("Change"):  
        if new_pwd == confirm_pwd and old_pwd:  
            st.success("Password changed successfully!")  
        else:  
            st.error("Passwords do not match or old password is empty.")
```

c. Job Posting

Enables students to apply for job:

```
# --- Job Posting for Companies ---
```

```
import pandas as pd
```

```
def company_page():
```

```
    st.title(" Company Dashboard")
```

```
    menu = ["View Applications", "Post Job", "Update Details", "Change Password"]
```

```
    choice = st.sidebar.selectbox("Select Action", menu)
```

```
    if choice == "View Applications":
```

```
        st.subheader(" Applications Received")
```

```
        applications = pd.DataFrame({
```

```
            "Student Name": ["Tanmay", "Aisha"],
```

```
            "Job ID": [101, 101],
```

```
            "Resume Link": ["link1.pdf", "link2.pdf"],
```

```
            "Status": ["Pending", "Reviewed"]
```

```
        })
```

```
        st.table(applications)
```

```
            st.success(f"Job '{job_title}' posted successfully!")
```

```
    elif choice == "Update Details":
```

```
        with st.form("update_details"):
```

```
            name = st.text_input("Company Name")
```

```
            email = st.text_input("Email")
```

```
            phone = st.text_input("Phone Number")
```

```
            submit = st.form_submit_button("Update")
```

```

if submit:
    st.success("Details updated successfully!")

elif choice == "Change Password":
    with st.form("change_password"):
        current_pwd = st.text_input("Current Password", type="password")
        new_pwd = st.text_input("New Password", type="password")
        confirm_pwd = st.text_input("Confirm New Password", type="password")
        submit = st.form_submit_button("Change Password")
    if submit:
        if new_pwd == confirm_pwd:
            st.success("Password changed successfully!")
        else:
            st.error("Passwords do not match.")

```

Advantages:-To ensure maintainability, modularity, and scalability of the Campus Recruitment System, the original implementations were refactored following object-oriented and clean code principles. The key focus was to separate data representation from business logic, enhance validation, and introduce reusable components:

7. Software Testing

To ensure the reliability and correctness of the refactored modules, structured testing strategies were applied. Testing covered both functional and non-functional aspects using white-box methods such as **Boundary Value Analysis (BVA)**, **Robust Testing**, **Worst Case Testing**, and **Decision Table Testing**.

7.1 Testing Approaches and Methodologies

The following testing methodologies were adopted:

- **Boundary Value Testing:** Tested inputs at their valid and invalid boundary ranges.
- **Robustness Testing:** Verified system behavior when receiving invalid, null, or unexpected inputs.
- **Worst Case Testing:** Combined all input variable extremes to identify corner case failures.
- **Decision Table Testing:** Evaluated complex logic paths based on combinations of multiple conditions.

7.2 Performing Testing on various test cases

Student Module:-

1. Testing for Student Application for Job

Test Case ID	Input Job ID	Resume Uploaded	Expected Result
1	101	Yes	Application submitted
2	(empty)	Yes	Warning shown
3	101	No	Warning shown
4	None	None	Input required

2. Testing for Change Password

Test Case ID	Old Password	New Password	Confirm Password	Expected Result
1	correct	newpass	newpass	Password changed
2	correct	newpass	wrongconfirm	Error: mismatch
3	(empty)	newpass	newpass	Error: old empty

3. Testing for Login

Test ID	Username	Password	Role	Expected Output
WC_01	student	student123	student	Login successful
WC_02	wrong	student123	student	Invalid credentials
WC_03	student	(empty)	student	Invalid credentials
WC_04	(empty)	(empty)	student	Invalid credentials

4. Decision Table Testing – Job Posting

Condition	Rule 1	Rule 2	Rule 3	Rule 4
Job Title is entered	Yes	No	Yes	Yes
Description is entered	Yes	Yes	No	Yes
Location is entered	Yes	Yes	Yes	No
Submit clicked	Yes	Yes	Yes	Yes
Expected Result	Pass	Fail	Fail	Fail

CONCLUSION

The Campus Recruitment System offers a contemporary and effective digital solution to streamline the campus placement procedure. By consolidating the engagements of students, recruiters, and administrators, the system improves coordination, transparency, and efficiency, thereby decreasing dependence on manual methods. The platform enhances recruitment processes, providing prompt updates, precise data handling, and a better experience for all participants. Upcoming advancements will enhance the system's functionalities, such as compatibility with additional platforms, the addition of real-time chat options, sophisticated machine learning for tailored job suggestions, and support for deployment across multiple institutions. These improvements seek to boost the system's scalability, accessibility, and overall worth to educational institutions and recruiting entities.

REFERENCES

1. Smith, J. (2023). Modern Campus Recruitment: Digitizing the Placement Process. *Journal of Educational Technology*, 15(3), 45- 60.
2. Kumar, A., & Sharma, P. (2022). Streamlining Recruitment in Higher Education Institutions: Challenges and Solutions. *Proceedings of the International Conference on Education and Technology*, 72-80.
3. Choudhury, R., & Das, M. (2021). Leveraging Machine Learning for Job Matching in Campus Recruitment Systems. *International Journal of Data Science and Analytics*, 12(2), 97-109.
4. Patel, R., & Verma, S. (2020). Building Scalable and Efficient Campus Recruitment Systems: A Case Study Approach. *Journal of Computer Applications in Education*, 10(4), 33-48.